Assignment 5 - Hosting custom Docker Images

by

Group 18

Christopher Sanden

Fabian Haukedal Johansen

Henrik Høie Corneliussen

Teodor Salvesen

in

IKT206

DevOps

Faculty of Technology and Science

University of Agder

Grimstad, March 2026

Contents

1 Introduction **1**

2 Prepare the code **2**
2.1 Starting point 2
2.2 Environment-specific database configuration 2
2.3 Changes in `Program.cs` 3
2.4 Migrations 6
2.5 Problems encountered 6

3 Docker and Docker Compose **7**
3.1 Dockerfile 7
3.2 Docker Compose 8
3.3 Caddy configuration 9
3.4 Problems encountered 10

4 Server deployment **11**
4.1 Overview 11
4.2 Domain and DNS 11
4.3 Preparing the VPS 12
4.4 Reverse SSH tunnel for repository access 12
4.5 Cloning the project 13
4.6 Starting the application stack 14
4.7 Verification 15
4.8 Problems encountered 16

5 Conclusion **17**

List of Figures

1 DNS A-record configuration for `devops.softsand.it`. 11
2 VPS prepared for deployment with Docker installed. 12
3 Reverse SSH tunnel from the school VM to the VPS. 13
4 Successful cloning of the project on the VPS. 13
5 Running Docker containers on the VPS after deployment. 14
6 The deployed application running publicly at `devops.softsand.it`. 15

List of Listings

1 Development configuration using SQLite 2
2 Production configuration using PostgreSQL 3
3 Relevant parts of `Program.cs` (part 1) 4
4 Relevant parts of `Program.cs` (part 2) 5
5 PostgreSQL migrations 6
6 SQLite migrations 6
7 Dockerfile used for the ASP.NET application 7
8 Docker Compose configuration 8
9 Production Caddyfile on the VPS 9
10 Reverse SSH tunnel used to expose the internal Gitea service to the VPS 12
11 Clone command used on the VPS 13
12 Docker Compose command used to deploy the application 14

1 Introduction

This report documents the preparation, containerisation, and deployment of an ASP.NET application to a public VPS with HTTPS. The assignment was divided into three main parts: adapting the existing project for production use, packaging the application with Docker and Docker Compose, and deploying the finished solution to a VPS connected to a public domain.

The main technical goal was not to change the functionality of the application itself, but to make it deployable in a realistic production setting. The starting point was a simple ASP.NET MVC application provided by the lecturer. The application originally used SQLite only and was configured primarily for development use. Our task was therefore to extend the configuration so that the application could still use SQLite in development, while using PostgreSQL in production.

The second part of the work focused on containerisation. A Dockerfile was created for the ASP.NET application, and Docker Compose was used to define the full stack consisting of the web application, a PostgreSQL database, and a Caddy reverse proxy. Caddy was used to expose the application publicly and handle HTTPS.

The final part of the assignment was deployment. A domain was purchased, a VPS was rented, and the application was deployed there using Docker Compose. Because the university Gitea server was not reachable from the public internet, a reverse SSH tunnel was required so that the VPS could temporarily reach the repository and clone the project. After cloning, the tunnel was no longer needed, and the application ran directly on the VPS.

The report explains the code changes made to support the production database, the Docker and Docker Compose configuration, and the steps taken to deploy the application publicly. It also documents the main problems encountered and how they were resolved.

2 Prepare the code

2.1 Starting point

The project provided by the lecturer was a simple ASP.NET MVC application. Functionally, the application itself was not the focus of the assignment. It was mainly used as a small database-backed web application suitable for deployment and infrastructure work. The original version was configured for development and used SQLite only.

Because the assignment required PostgreSQL and migrations in production mode, the application had to be modified so that it could use one database provider in development and another in production. The goal was to keep local development simple while making the production deployment realistic and compatible with Docker Compose.

2.2 Environment-specific database configuration

The most important code change was the introduction of environment-specific database configuration. In development, the application was configured to use SQLite through `appsettings.Development.json`. In production, the application used PostgreSQL through the default production configuration in `appsettings.json`.

The final development configuration used SQLite with the following connection string:

```
{
  "Database": {
    "Provider": "Sqlite"
  },
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Listing 1: Development configuration using SQLite

The production configuration used PostgreSQL with the following connection string:

```
{
  "Database": {
    "Provider": "Postgres"
  },
  "ConnectionStrings": {
    "DefaultConnection": "Host=db;Port=5432;Database=db;Username=user;Password=password"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Listing 2: Production configuration using PostgreSQL

One important detail was that the production connection string could not use `localhost` as the database host once the application ran inside Docker. Inside the web container, `localhost` refers to the container itself, not the PostgreSQL container. The host therefore had to be set to `db`, which is the Docker Compose service name for the database container.

2.3 Changes in `Program.cs`

`Program.cs` was updated so that the application selected the correct database provider depending on the environment. The final implementation distinguished explicitly between development and non-development environments, validated that the configured provider matched the expected one, and registered the correct DbContext type accordingly.

```
using Example.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string 'DefaultConnection' was not found.");
var configuredProvider = builder.Configuration["Database:Provider"]
    ?? throw new InvalidOperationException("Database provider configuration 'Database:Provider' was not found.");

if (builder.Environment.IsDevelopment())
{
    EnsureProviderMatchesEnvironment(configuredProvider, expectedProvider: "Sqlite", builder.Environment.EnvironmentName);
    ValidateConnectionString(connectionString, configuredProvider);

    builder.Services.AddDbContext<ApplicationDbContext, SqliteApplicationDbContext>(options =>
        options.UseSqlite(connectionString));
}
else
{
    EnsureProviderMatchesEnvironment(configuredProvider, expectedProvider: "Postgres", builder.Environment.EnvironmentName);
    ValidateConnectionString(connectionString, configuredProvider);

    builder.Services.AddDbContext<ApplicationDbContext, PostgresApplicationDbContext>(options =>
        options.UseNpgsql(connectionString));
}

builder.Services.AddDatabaseDeveloperPageExceptionFilter();


var identityBuilder = builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true);
if (builder.Environment.IsDevelopment())
{
    identityBuilder.AddEntityFrameworkStores<SqliteApplicationDbContext>();
}
else
{
    identityBuilder.AddEntityFrameworkStores<PostgresApplicationDbContext>();
}

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

Listing 3: Relevant parts of `Program.cs` (part 1)

```

using (var services = app.Services.CreateScope())
{
    var db = services.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    db.Database.Migrate();

    if (app.Environment.IsDevelopment())
    {
        ApplicationDbInitializer.SeedDevelopmentData(db);
    }
}

if (app.Environment.IsDevelopment())
{
    app.UseMigrationsEndPoint();
}
else
{
    app.UseExceptionHandler("/Home/Error");
    // app.UseHsts();
}

// app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapRazorPages();

app.Run();
```

Listing 4: Relevant parts of `Program.cs` (part 2)

In development, the application registers `SqliteApplicationDbContext` and uses SQLite. In production, it registers `PostgresApplicationDbContext` and uses PostgreSQL. This separation was cleaner than trying to reuse the same configuration for both environments and made it easier to keep migrations organised.

Additional validation methods were also introduced so that the application could fail early if the wrong database provider or an obviously incorrect connection string was used in a given environment. This made it easier to catch configuration mistakes before runtime.

2.4 Migrations

Separate migrations were created for SQLite and PostgreSQL. This ensured that each provider had its own migration history and model snapshot. The migration folders were organised under `Data/Migrations/Sqlite` and `Data/Migrations/Postgres`.

The PostgreSQL migrations identified in the project were:

```
20260327180444_InitialPostgres
20260329123153_DateOnlyColumns
```

Listing 5: PostgreSQL migrations

The SQLite migrations identified in the project were:

```
20260327180913_InitialSqlite
20260329123145_DateOnlyColumns
```

Listing 6: SQLite migrations

During application startup, `db.Database.Migrate()` was executed so that the correct schema could be applied automatically. In development mode, development data was also seeded after migration. This made it possible to recreate the database schema automatically when containers were built from scratch.

2.5 Problems encountered

The main code-related problem was the production connection string. At one point, the application attempted to connect to PostgreSQL using `localhost`, which caused the containerised application to fail because the PostgreSQL server was running in a separate container. Once the host was changed to `db`, the web container was able to communicate with the database correctly.

In summary, the code preparation stage did not involve significant functional changes to the website itself. Instead, it focused primarily on making the project environment-aware, supporting separate development and production databases, and ensuring that the schema could be created automatically through migrations.

3 Docker and Docker Compose

3.1 Dockerfile

The ASP.NET application was containerised using a multi-stage Dockerfile. This made it possible to build and publish the application in a full .NET SDK image, while running the final application in a smaller ASP.NET runtime image. This approach reduced the size of the final image and is standard practice for .NET container deployments.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build-env
WORKDIR /app

COPY Example.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o out

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app

COPY --from=build-env /app/out .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "Example.dll"]
```

Listing 7: Dockerfile used for the ASP.NET application

The first stage uses the .NET 8 SDK image and prepares the build environment. The working directory is set to `/app`, and the project file is copied in first so that dependency restore can be cached. After this, the rest of the source code is copied into the image and the application is published in Release mode to the `out` directory.

The second stage uses the smaller ASP.NET runtime image. The published output from the build stage is copied into the runtime image, and the application is configured to listen on port 8080 over HTTP. This is important because HTTPS is handled by Caddy rather than by the ASP.NET application itself. Finally, the container starts the application by running `Example.dll`.

3.2 Docker Compose

Docker Compose was used to define the complete production stack. The final setup consisted of three services: the ASP.NET web application, a PostgreSQL database, and a Caddy reverse proxy. This made it possible to build and start the entire application with a single command.

```yaml
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: web
    environment:
      ASPNETCORE_ENVIRONMENT: Production
    depends_on:
      - db
    expose:
      - "8080"

  db:
    image: postgres:16
    container_name: db
    environment:
      POSTGRES_DB: db
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    expose:
      - "5432"

  proxy:
    image: caddy:2
    container_name: proxy
    depends_on:
      - web
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - caddy_data:/data
      - caddy_config:/config
volumes:
  postgres_data:
  caddy_data:
  caddy_config:
```

Listing 8: Docker Compose configuration

The `web` service builds the ASP.NET application from the Dockerfile in the project root. It sets the environment to Production so that the application uses the PostgreSQL configuration. Port 8080 is exposed internally so that other services, especially the reverse proxy, can reach it.

The `db` service uses the official PostgreSQL 16 image. The environment variables define the database name, user, and password. A named volume is used to preserve the database data between container restarts. Port 5432 is exposed internally to the Docker network.

The `proxy` service uses the official Caddy image. It depends on the web service and maps ports 80 and 443 to the host, allowing the application to be reached publicly over HTTP and HTTPS. The Caddyfile is mounted into the container, and named volumes are used for Caddy's persistent data and configuration.

The named volumes at the end of the file ensure persistent storage for PostgreSQL and for Caddy's TLS-related state.

3.3 Caddy configuration

Caddy was used as the public reverse proxy because it supports a simple configuration model and automatic HTTPS. During local testing, a simpler HTTP-only configuration was used. For the final deployment on the VPS, the domain-based configuration below was used.

```
devops.softsand.it {
    reverse_proxy web:8080
}
```

Listing 9: Production Caddyfile on the VPS

This configuration instructs Caddy to accept requests for `devops.softsand.it` and forward them to the `web` container on port 8080 inside the Docker network. Because HTTPS was terminated by Caddy, the ASP.NET application itself only needed to serve HTTP internally.

3.4 Problems encountered

During development of the Docker setup, several issues had to be resolved. One problem was malformed YAML in the Compose file, which prevented Docker Compose from parsing the configuration correctly. Another problem was database startup and connectivity, especially while the web application was still using an incorrect host name for PostgreSQL.

HTTPS and reverse proxy behaviour also caused problems during testing. In particular, attempts to mix temporary tunnel-based testing with domain-based proxy configuration led to redirect behaviour that did not reflect the final production architecture. Once the final deployment was simplified to a direct VPS-based setup, these issues became much easier to resolve.

4 Server deployment

4.1 Overview

The final deployment target was a VPS hosted by Hetzner running Ubuntu 24.04.4 LTS. The complete application stack was run directly on the VPS using Docker Compose. This included the ASP.NET web application, the PostgreSQL database, and the Caddy reverse proxy.

The public domain used for the deployment was `devops.softsand.it`. The domain was configured so that it pointed directly to the VPS, allowing Caddy to serve the application publicly over HTTPS.

4.2 Domain and DNS

A subdomain was configured so that `devops.softsand.it` resolved to the public IP address of the VPS. This step was necessary before the application could be accessed publicly and before Caddy could successfully handle HTTPS for the deployed service.

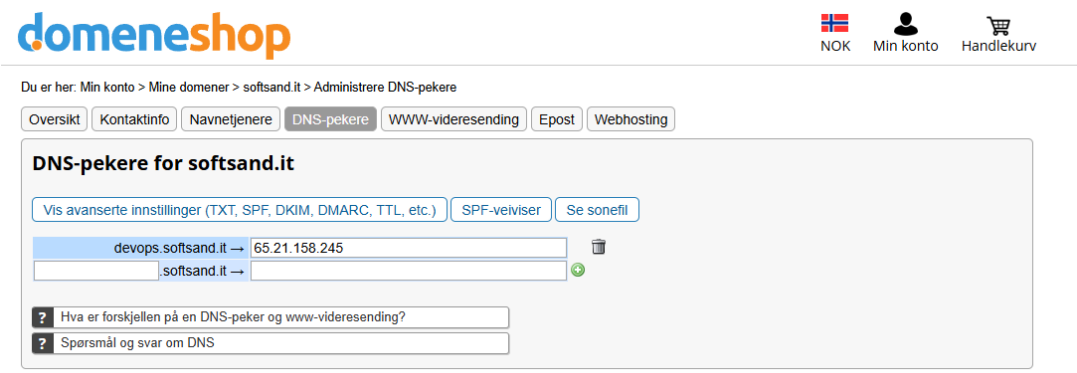


Figure 1: DNS A-record configuration for `devops.softsand.it`.

4.3 Preparing the VPS

After access to the VPS had been obtained, the server was prepared for deployment. Docker and Docker Compose were installed manually on the server. The installation followed Docker's official Ubuntu installation guide. In addition, ports 80 and 443 had to be available so that HTTP and HTTPS traffic could reach Caddy.

The VPS was then ready to receive the application source code and run the full stack directly through Docker Compose.

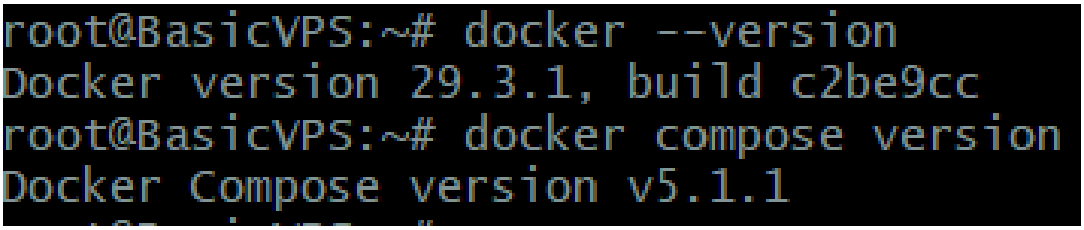


Figure 2: VPS prepared for deployment with Docker installed.

4.4 Reverse SSH tunnel for repository access

The Gitea server used for the project was not reachable directly from the public internet. Because of this, the VPS could not clone the repository by contacting the Gitea server directly. A reverse SSH tunnel was therefore used to make the internal Gitea service temporarily reachable from the VPS.

The reverse SSH command used on the school VM was:

```
ssh -N -R 3000:localhost:3000 root@65.21.158.245
```

Listing 10: Reverse SSH tunnel used to expose the internal Gitea service to the VPS

This command opened port 3000 on the VPS and forwarded traffic on that port to port 3000 on the school VM, where the Gitea service was reachable. In practice, this allowed the VPS to access the repository through `http://localhost:3000` during the cloning step.

It is important to note that the reverse SSH tunnel was used only to make the repository reachable long enough to perform the clone. It was not part of the final runtime architecture. Once the repository had been cloned successfully, the application itself ran directly on the VPS.

```
christops@ikt206g26v-christops:~/gitea$ ssh -N -R 3000:localhost:3000 root@65.21.158.245
```

Figure 3: Reverse SSH tunnel from the school VM to the VPS.

4.5 Cloning the project

Once the reverse SSH tunnel was active, the repository could be cloned directly on the VPS using the forwarded local address. The clone command used was:

```
git clone http://localhost:3000/Chris/ikt206g26v-05.git
```

Listing 11: Clone command used on the VPS

After the repository had been cloned, the reverse SSH tunnel was no longer required. At that point, the VPS contained the full project and could be used as the sole deployment target.

```
root@BasicVPS:~# git clone http://localhost:3000/Chris/ikt206g26v-05.git
Cloning into 'ikt206g26v-05'...
remote: Enumerating objects: 545, done.
remote: Counting objects: 100% (545/545), done.
remote: Compressing objects: 100% (326/326), done.
remote: Total 545 (delta 179), reused 493 (delta 154), pack-reused 0 (from 0)
Receiving objects: 100% (545/545), 28.27 MiB | 16.74 MiB/s, done.
Resolving deltas: 100% (179/179), done.
```

Figure 4: Successful cloning of the project on the VPS.

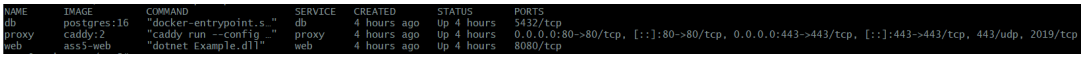
4.6 Starting the application stack

After changing into the project directory on the VPS, the application stack was started using Docker Compose. The deployment command used was:

```
docker compose up --build -d
```

Listing 12: Docker Compose command used to deploy the application

This command built the ASP.NET image, created the PostgreSQL and Caddy containers, and started the full stack in the background. The running containers could then be inspected using standard Docker Compose commands such as `docker compose ps`.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
db	postgres:16	"docker-entrypoint.s…"	db	4 hours ago	Up 4 hours	5432/tcp
proxy	caddy:2	"caddy run --config …"	proxy	4 hours ago	Up 4 hours	0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
web	ass5-web	"dotnet Example.dll"	web	4 hours ago	Up 4 hours	8080/tcp

Figure 5: Running Docker containers on the VPS after deployment.

4.7 Verification

The deployment was verified by checking that the containers were running correctly and by opening the public website in a browser. The final working URL was: https://devops.softsand.it

Once the website was reachable through the public domain, the deployment could be considered complete. This confirmed that the DNS configuration, Docker Compose deployment, PostgreSQL integration, and Caddy HTTPS setup were all functioning together correctly on the VPS.

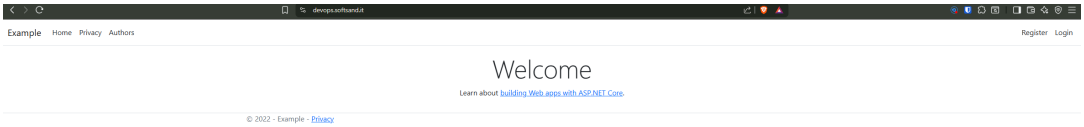


Figure 6: The deployed application running publicly at `devops.softsand.it`.

4.8 Problems encountered

The most important deployment issue was an initial misunderstanding of the role of the reverse SSH tunnel. At first, it appeared possible that the tunnel might be needed as part of the permanent runtime setup. A closer reading of the assignment made it clear that this was not the case. The tunnel was only required so that the VPS could temporarily reach the internal Gitea server and clone the project.

Other problems included database connectivity mistakes inside Docker, YAML syntax mistakes in the Compose file, and redirect-related behaviour while testing different combinations of HTTPS, Caddy, and temporary tunnel-based routing. These issues were resolved by simplifying the final architecture and running the complete stack directly on the VPS.

5 Conclusion

In this assignment, we adapted an ASP.NET application so that it could use SQLite in development and PostgreSQL in production. The project was then containerised using a multi-stage Dockerfile and a Docker Compose configuration containing the web application, a PostgreSQL database, and a Caddy reverse proxy.

The final deployment was completed on a Hetzner VPS running Ubuntu 24.04.4 LTS. A public subdomain was configured to point to the server, and Caddy was used to expose the application over HTTPS. Because the Gitea server was not publicly reachable, a reverse SSH tunnel was used during the cloning phase so that the VPS could temporarily access the internal repository. After cloning, the tunnel was no longer needed, and the application ran directly on the VPS.

The main difficulties during the work were related to configuration rather than application logic. In particular, Docker networking, environment-specific database settings, and reverse proxy behaviour required careful adjustment. The most important fix was changing the production PostgreSQL host from `localhost` to the Docker service name `db`. Once these issues had been resolved, the deployment worked as intended and the website became publicly available at `https://devops.softsand.it`.

Overall, the assignment demonstrated the full workflow from preparing an application for production to deploying it publicly with containers, a production database, and HTTPS.